

Ladno Ok's Org
Scan Report



Application: tiwap
Environment: Development
Completed On: Tue, 17 Feb 2026 17:03:45 GMT

Application: tiwap
Environment: Development

Scan Date: Tue, 17 Feb 2026 17:03:45 GMT | Paths Scanned: 35 | Scan Duration: 75s

High	Medium	Low	New	Assigned	Risk Accepted	False Positive
7	35	34	76	0	0	0

Criticality	CWE	Finding	New	Assigned	Risk Accepted	False Positive
High	CWE-918	Server Side Request Forgery	2	0	0	0
High	CWE-89	SQL Injection	3	0	0	0
High	CWE-79	Cross Site Scripting (Reflected)	1	0	0	0
High	CWE-98	Remote File Inclusion	1	0	0	0
Medium	CWE-352	Anti CSRF Tokens Scanner	10	0	0	0
Medium	CWE-693	Content Security Policy (CSP) Header Not Set	13	0	0	0
Medium	CWE-1021	Missing Anti-clickjacking Header	12	0	0	0
Low	CWE-693	X-Content-Type-Options Header Missing	12	0	0	0
Low	CWE-829	Cross-Domain JavaScript Source File Inclusion	6	0	0	0
Low	CWE-200	Server Leaks Version Information via "Server" HTTP Response Header Field	16	0	0	0

Server Side Request Forgery

Criticality: High CWE ID: 918

Description:

Remediation

To mitigate the vulnerability of Server Side Request Forgery, the following steps can be taken:

- 1. Input validation:** Validate and sanitize all user-supplied input to ensure that it conforms to expected formats and does not contain any malicious content. This includes validating URLs and ensuring they are pointing to the expected destinations.
- 2. Whitelisting:** Implement a whitelist of allowed URLs or IP addresses that the server is allowed to access. This can help prevent requests to unauthorized or potentially malicious destinations.
- 3. Use of safe APIs:** Utilize safe APIs or libraries that provide built-in protection against Server Side Request Forgery. These APIs can handle URL validation and ensure that requests are being sent to the intended destinations.
- 4. Restrict server permissions:** Limit the permissions of the server to only access necessary resources and restrict access to sensitive information. This can help minimize the impact of a successful Server Side Request Forgery attack.

About

Server Side Request Forgery (SSRF) is a vulnerability where an attacker can manipulate the server to make requests to arbitrary URLs or IP addresses. The web server receives a remote address and retrieves the contents of this URL, but it does not sufficiently ensure that the request is being sent to the expected destination. This can lead to various security risks, including unauthorized access to internal resources, information disclosure, and potential remote code execution.

Risks

The risks associated with Server Side Request Forgery include:

- 1. Unauthorized access:** An attacker can exploit SSRF to make requests to internal resources that are not intended to be accessible from the outside. This can lead to unauthorized access to sensitive information or systems.
- 2. Information disclosure:** By manipulating the server to make requests to external URLs, an attacker can potentially retrieve sensitive information from those URLs and disclose it to

unauthorized parties.

3. Remote code execution: In some cases, SSRF can be used to execute arbitrary code on the server by making requests to URLs that contain malicious payloads. This can lead to complete compromise of the server and potentially the entire system.

It is important to address and remediate SSRF vulnerabilities to prevent these risks and protect the integrity and security of the web application and server.

Path	Method	Status
/api/stock/product?product=http://192.168.107.7:35779/bd31368a-6dd0-4525-9c3b-a7fda5c55f54	GET	NEW
/ssrf	POST	NEW

SQL Injection

Criticality: High CWE ID: 89

Description:

Remediation

- 1. SQL ORM Equivalent (Using Sequelize in Node.js)** Instead of writing raw SQL queries and risking SQL injection, ORMs like Sequelize allow you to define models that represent your database tables.

```
const { Sequelize, DataTypes } = require('sequelize');
const sequelize = new Sequelize('database', 'username', 'password', {
  host: 'localhost',
  dialect: 'postgres', // or 'mysql', 'sqlite', etc.
});
// Define the User model
const User = sequelize.define('User', {
  username: {
    type: DataTypes.STRING,
    allowNull: false
  },
  password: {
    type: DataTypes.STRING,
    allowNull: false
  }
});
// Sanitize and save user input
async function registerUser(req, res) {
  try {
    const newUser = await User.create({
      username: req.body.username.trim(),
      password: req.body.password.trim()
    });
    res.status(201).json(newUser);
  } catch (error) {
    res.status(500).json({ error: 'User creation failed' });
  }
}
```

- 2. Use parameterized queries:** Instead of concatenating user input directly into SQL statements, use parameterized queries provided by the language framework. Parameterized queries separate the SQL code from the user input, preventing SQL injection attacks. For example, in a Java application using JDBC, you can use prepared statements:

```
String sql = "SELECT * FROM users WHERE username = ?";
PreparedStatement statement = connection.prepareStatement(sql);
statement.setString(1, sanitize(userInput));
ResultSet resultSet = statement.executeQuery();
```

3. Avoid string concatenation on SQL statements: String concatenation should be avoided when constructing SQL statements, as it can lead to SQL injection vulnerabilities. Instead, use parameterized queries or stored procedures to handle dynamic values. For example, in a PHP application using PDO:

```
$username = sanitize($_POST['username']);
$password = sanitize($_POST['password']);
$sql = "SELECT * FROM users WHERE username = :username AND password = :password";
$stmt = $pdo->prepare($sql);
$stmt->bindParam(':username', $username);
$stmt->bindParam(':password', $password);
$stmt->execute();
```

About

SQL injection vulnerabilities occur when user input is not properly sanitized or when it is directly passed to the back-end SQL server. Attackers can exploit these vulnerabilities by injecting SQL commands into the application, potentially allowing them to enumerate column names, retrieve data from the database, or even execute remote code.

Risks

The risks associated with SQL injection vulnerabilities include:

- Enumeration of column names:** Attackers can use SQL injection to gather information about the database structure, such as the names of tables and columns. This information can be used for further attacks or unauthorized access.
- Data retrieval:** SQL injection can allow attackers to retrieve sensitive data from the database, such as usernames, passwords, or personal information.
- Remote code execution:** In some cases, SQL injection can lead to remote code execution, where an attacker can execute arbitrary code on the server, potentially gaining full control over the application and the underlying system.

It's important to note that StackHawk tests for SQL injection vulnerabilities are often time-based, meaning the scanner tries to make the SQL server perform time-based actions to increase the response time of the application. This can help identify potential SQL injection vulnerabilities.

Path	Method	Status
/business-logic	POST	NEW
/reflected-xss	POST	NEW
/xxe?name=HSTE AND 1=1 --	GET	NEW

Cross Site Scripting (Reflected)

Criticality: High CWE ID: 79

Description:

Remediation

- 1. Filtering allowable data:** Implement a mechanism to filter and validate user input to ensure that only expected and safe data is accepted. This can be done by using input validation techniques such as regular expressions or whitelist filtering.
- 2. Encoding data:** Use proper encoding techniques to ensure that user input is properly encoded before being stored in the backend system or presented in the DOM. This includes encoding data in HTML, URL, CSS, or JavaScript as per the context in which it is being used.
- 3. Content Security Policy (CSP):** Implement a content security policy that restricts the types of content that can be loaded and executed on a web page. This can help prevent XSS attacks by blocking the execution of malicious scripts.

Example of input validation in a web application using PHP:

```
$input = $_POST['input'];  
if (preg_match('/^[a-zA-Z0-9]+$/', $input)) {  
    // Input is valid, proceed with further processing  
} else {  
    // Input is not valid, display an error message  
}
```

Example of encoding data in a web application using JavaScript:

```
var userInput = document.getElementById('userInput').value;  
var encodedInput = encodeURIComponent(userInput);  
document.getElementById('output').innerHTML = encodedInput;
```

Example of setting a content security policy in a web application using HTTP headers:

```
Content-Security-Policy: default-src 'self'; script-src 'self' 'unsafe-  
inline'; style-src 'self' 'unsafe-inline'
```

About

Cross-site scripting issues occur when user input is not properly validated or encoded before being stored in a backend system or presented back within the DOM. This vulnerability allows attackers to inject malicious scripts into web pages, potentially compromising the security of the application and its users.

Risks

Exploiting cross-site scripting vulnerabilities can lead to various risks, including:

- 1. Information disclosure:** Attackers can use XSS to obtain sensitive information about other users of the system, such as session cookies or tokens. This information can be used for account takeover or other malicious activities.

2. **Phishing attacks:** XSS can be used to trick users into revealing their credentials or other sensitive information by presenting them with a fake login form or other deceptive content.
3. **Malware distribution:** Attackers can inject malicious scripts that can download and execute malware on the victim's system, potentially leading to further compromise or control of the system.

Path	Method	Status
/reflected-xss	POST	NEW

Remote File Inclusion

Criticality: High CWE ID: 98

Description:

Remediation

To mitigate the risk of Remote File Inclusion (RFI) attacks, web application developers should implement the following security measures:

- 1. Input validation:** Validate and sanitize all user-supplied input, including URL parameters and file paths, to ensure they conform to expected formats and do not contain any malicious code or characters. Use server-side validation techniques such as regular expressions or whitelist-based filtering to validate input.
- 2. Avoid dynamic file inclusion:** Whenever possible, avoid using user-supplied input to construct file paths for inclusion. Instead, use static file paths or predefined variables to reference include files. This reduces the risk of including remote files with malicious code.
- 3. Use whitelisting:** Maintain a whitelist of allowed file paths and only include files from this list. This prevents the inclusion of arbitrary files from external sources.
- 4. Disable remote file inclusion:** If remote file inclusion is not required for the application's functionality, disable it entirely. This can be done by configuring the server or application framework to disallow the inclusion of files from remote sources.
- 5. Implement access controls:** Restrict access to sensitive files and directories by setting appropriate file permissions and using access control mechanisms provided by the operating system or web server. This helps prevent unauthorized access to critical files.
- 6. Regularly update and patch:** Keep the web application framework, server software, and any third-party libraries up to date with the latest security patches. Vulnerabilities in these components can be exploited to bypass security measures and execute remote file inclusion attacks.

About

Remote File Inclusion (RFI) is an attack technique that targets web applications with "dynamic file include" mechanisms. It exploits the ability of web applications to include external files based on user-supplied input, such as URLs or parameter values. By manipulating this input, an attacker can trick the web application into including remote files containing malicious code.

Web application frameworks commonly support file inclusion to modularize code and improve maintainability. However, if the choice of module to include is based on elements from the HTTP request without proper validation, the application becomes vulnerable to RFI attacks.

PHP is particularly susceptible to RFI attacks due to its extensive use of file includes and default server configurations that increase vulnerability.

Risks

Remote File Inclusion (RFI) attacks pose several risks to web applications and their users:

- 1. Server compromise:** By including malicious files, an attacker can execute arbitrary code on the server. If the file inclusion is not properly wrapped or executed, the code in the included files runs in the context of the server user, potentially leading to a complete system compromise.
- 2. Client compromise:** Attackers can manipulate the content of the response sent to clients, embedding malicious code such as JavaScript. When executed by the client's browser, this code can steal sensitive information, such as session cookies, leading to unauthorized access to user accounts.
- 3. Data leakage:** RFI attacks can allow attackers to access and retrieve sensitive data stored on the server. This includes confidential user information, intellectual property, or other sensitive data that may be accessible through the compromised server.
- 4. Application integrity:** By including malicious files, attackers can modify the behavior of the web application, potentially altering or deleting data, defacing the website, or disrupting its normal operation.

It is crucial for web application developers to understand and address the risks associated with RFI attacks to ensure the security and integrity of their applications and protect user data.

Path	Method	Status
/api/stock/product?product=http://www.google.com/	GET	NEW

Anti CSRF Tokens Scanner

Criticality: Medium CWE ID: 352

Description:

Remediation

To mitigate the risk of the "Anti CSRF Tokens Scanner" vulnerability, the following steps can be taken:

- 1. Implement CSRF tokens:** Use anti-CSRF tokens in all forms and AJAX requests to ensure that each request is accompanied by a unique token. This token should be generated on the server-side and included in the form or request payload. On the server-side, validate the token for each incoming request to ensure its authenticity.

Example in PHP:

```
// Generate CSRF token and include it in the form
$csrfToken = bin2hex(random_bytes(32));
echo '<input type="hidden" name="csrf_token" value="' . $csrfToken .
'">';
// Validate CSRF token on the server-side
if ($_POST['csrf_token'] !== $_SESSION['csrf_token']) {
// Invalid token, handle the error
}
```

- 2. Use SameSite cookies:** Set the SameSite attribute for cookies to prevent them from being sent in cross-site requests. This attribute restricts the cookie to be sent only with requests originating from the same site.

Example in JavaScript:

```
document.cookie = 'session_id=123; SameSite=Strict';
```

- 3. Implement strict referer checking:** Validate the Referer header of incoming requests to ensure that they originate from the same site. This can help detect and block requests from unauthorized sources.

Example in Apache .htaccess:

```
RewriteEngine On
RewriteCond %{HTTP_REFERER} !^https://www.example.com/ [NC]
RewriteRule ^ - [F]
```

About

Cross-Site Request Forgery (CSRF) is an attack that exploits the trust between a website and its users. It involves tricking a victim into unknowingly sending a malicious request to a target website, using the victim's authenticated session or network connection. The attack takes advantage of predictable URL/form actions and can be performed even if the victim is not aware of it.

CSRF attacks can be prevented by implementing measures such as CSRF tokens, SameSite cookies, and strict referer checking. These techniques ensure that each request is accompanied by a unique token, restrict the sending of cookies in cross-site requests, and validate the referer header to ensure requests originate from the same site.

Risks

The "Anti CSRF Tokens Scanner" vulnerability poses several risks:

- 1. Unauthorized actions:** An attacker can use CSRF to perform actions on behalf of the victim without their knowledge or consent. This can lead to unauthorized changes, such as modifying account settings, making financial transactions, or deleting data.
- 2. Information disclosure:** CSRF attacks can also be used to gain access to sensitive information by exploiting vulnerabilities in the target site. If the target site is vulnerable to cross-site scripting (XSS), the risk of information disclosure is further increased, as XSS can be used as a platform for CSRF attacks.
- 3. Session hijacking:** CSRF attacks can hijack an active session or authentication credentials of the victim, allowing the attacker to impersonate the victim and perform actions with their privileges. This can lead to further compromise of the victim's account or unauthorized access to sensitive resources.

It is crucial to implement proper countermeasures to prevent CSRF attacks and protect both the website and its users from these risks.

Path	Method	Status
/blind-sql-injection	GET	NEW
/business-logic	GET	NEW

/cmd-injection	GET	NEW
/reflected-xss	GET	NEW
/sensitive-data-exposure	GET	NEW
/settings	GET	NEW
/settings	GET	NEW
/sql-injection	GET	NEW
/ssrf	GET	NEW
/ssrf?product=food&stock=17	GET	NEW

Content Security Policy (CSP) Header Not Set

Criticality: Medium CWE ID: 693

Description:

Remediation

To fix the "Content Security Policy (CSP) Header Not Set" vulnerability, you need to properly configure the Content Security Policy header in your web server or application. Here are the steps to remediate this issue:

1. Identify the web server or application: Determine the technology stack used for your website or application. This could be Apache, Nginx, IIS, or a specific web framework like Express.js or Django.

2. Configure the Content Security Policy header: Add the Content Security Policy header to your server or application configuration. The exact method will depend on the technology stack you are using.

- For Apache, add the following line to your .htaccess file or Apache configuration file:

```
Header set Content-Security-Policy "default-src 'self';"
```

- For Nginx, add the following line to your server block or location block:

```
add_header Content-Security-Policy "default-src 'self';";
```

- For IIS, open the IIS Manager, select your website, and go to the "HTTP Response Headers" section. Add a new header with the name "Content-Security-Policy" and the value "default-src 'self';".

- For specific web frameworks, consult the documentation for that framework on how to set the Content Security Policy header.

3. Customize the Content Security Policy: The example above sets a basic Content Security Policy that only allows resources to be loaded from the same origin. However, you may need to customize the policy based on your website's requirements. Refer to the Content Security Policy specification (<https://developer.mozilla.org/en-US/docs/Web/HTTP/CSP>) for more information on the available directives and their usage.

4. Test and monitor: After configuring the Content Security Policy header, thoroughly test your website or application to ensure that all resources are being loaded correctly. Monitor your application's security logs and reports to detect any potential violations or issues with the Content Security Policy.

About

Content Security Policy (CSP) is an added layer of security that helps to detect and mitigate certain types of attacks, including Cross-Site Scripting (XSS) and data injection attacks. These attacks can be used for various malicious purposes, such as data theft, site defacement, or distribution of malware.

CSP provides a set of standard HTTP headers that allow website owners to declare approved sources of content that browsers should be allowed to load on a page. The covered types of content include JavaScript, CSS, HTML frames, fonts, images, and embeddable objects like Java applets, ActiveX, audio, and video files.

By properly configuring the Content Security Policy header, you can restrict the loading of content from unauthorized sources and reduce the risk of XSS attacks and other malicious activities.

Risks

Failure to set the Content Security Policy (CSP) header exposes your website or application to various security risks:

- 1. Cross-Site Scripting (XSS) attacks:** Without a proper CSP, attackers can inject malicious scripts into your web pages, leading to XSS attacks. These attacks can result in the theft of sensitive user information, session hijacking, or defacement of your website.
- 2. Data injection attacks:** Attackers can inject malicious data into your website or application, leading to data corruption, unauthorized access, or the execution of arbitrary code.
- 3. Malware distribution:** Without a CSP, attackers can load and execute malicious files on your website, leading to the distribution of malware to your users.
- 4. Site defacement:** Attackers can modify the appearance or content of your website, damaging your brand reputation and causing user distrust.

By implementing a proper Content Security Policy, you can significantly reduce the risk of these security threats and protect your website or application from potential attacks.

Path	Method	Status
/blind-sql-injection	GET	NEW
/business-logic	GET	NEW
/cmd-injection	GET	NEW
/dashboard	GET	NEW
/index.css	GET	NEW
/reflected-xss	GET	NEW
/robots.txt	GET	NEW
/security-misconfig	GET	NEW
/sensitive-data-exposure	GET	NEW
/settings	GET	NEW
/sql-injection	GET	NEW
/xxe	GET	NEW

Missing Anti-clickjacking Header

Criticality: Medium CWE ID: 1021

Description:

Remediation

To fix the vulnerability of "Missing Anti-clickjacking Header", you need to include either the Content-Security-Policy header with the frame-ancestors directive or the X-Frame-Options header in the server response. These headers provide protection against ClickJacking attacks by specifying which origins are allowed to embed the page in a frame or iframe.

Content-Security-Policy Example:

```
Content-Security-Policy: frame-ancestors 'self' https://trusted-site.com;
```

In this example, the frame-ancestors directive is set to allow embedding the page only on the same origin ('self') and on the https://trusted-site.com origin.

X-Frame-Options Example:

```
X-Frame-Options: SAMEORIGIN
```

In this example, the X-Frame-Options header is set to SAMEORIGIN, which allows embedding the page only on the same origin.

About

The vulnerability "Missing Anti-clickjacking Header" occurs when the server response does not include either the Content-Security-Policy header with the frame-ancestors directive or the X-Frame-Options header. These headers are used to protect against ClickJacking attacks, where an attacker tricks a user into clicking on a malicious element hidden behind a legitimate element on a webpage.

Risks

Without the proper anti-clickjacking headers, an application is vulnerable to ClickJacking attacks. This can lead to various security risks, including:

- 1. Unauthorized actions:** An attacker can trick users into performing actions on the vulnerable page without their knowledge or consent. This can include clicking on buttons, submitting forms, or interacting with sensitive functionality.
- 2. Phishing attacks:** ClickJacking can be used to overlay legitimate-looking login forms or other sensitive information prompts on top of trusted websites. This can trick users into entering their credentials or personal information, which can then be captured by the attacker.
- 3. Malware installation:** ClickJacking can be used to trick users into unknowingly downloading and installing malware on their devices. By overlaying a legitimate-looking download prompt on

top of a trusted website, the attacker can deceive users into executing malicious files.

It is important to implement the appropriate anti-clickjacking headers to mitigate these risks and protect users from ClickJacking attacks.

Path	Method	Status
/blind-sql-injection	GET	NEW
/business-logic	GET	NEW
/cmd-injection	GET	NEW
/dashboard	GET	NEW
/reflected-xss	GET	NEW
/security-misconfig	GET	NEW
/sensitive-data-exposure	GET	NEW
/settings	GET	NEW
/sql-injection	GET	NEW
/ssrf	GET	NEW
/ssrf?product=food&stock=17	GET	NEW
/xxe	GET	NEW

X-Content-Type-Options Header Missing

Criticality: Low CWE ID: 693

Description:

Remediation

To fix this vulnerability, the X-Content-Type-Options header should be set to 'nosniff'. This will prevent older versions of Internet Explorer and Chrome from performing MIME-sniffing on the response body.

To set the X-Content-Type-Options header to 'nosniff', you can add the following configuration to your web server:

```
<IfModule mod_headers.c>  
Header set X-Content-Type-Options "nosniff"  
</IfModule>
```

If you are using a different web server, refer to its documentation for the specific configuration syntax.

About

The X-Content-Type-Options header is a security feature that helps prevent MIME-sniffing attacks. When this header is set to 'nosniff', it instructs the browser to strictly interpret the content type specified in the response headers and not perform any MIME-sniffing. This ensures that the response body is displayed as the intended content type and not interpreted as something else.

Without the X-Content-Type-Options header set to 'nosniff', older versions of Internet Explorer and Chrome may perform MIME-sniffing on the response body. This means that they may ignore the declared content type and try to guess the content type based on the actual content. This can lead to security vulnerabilities, as the browser may interpret and display the response body as a different content type, potentially exposing sensitive information or executing malicious code.

Risks

If the X-Content-Type-Options header is missing or not set to 'nosniff', there are several risks:

- 1. MIME-sniffing attacks:** Older versions of Internet Explorer and Chrome may perform MIME-sniffing on the response body, potentially causing it to be interpreted and displayed as a content type other than the declared content type. This can lead to security vulnerabilities and expose sensitive information.
- 2. Content type misinterpretation:** Without the X-Content-Type-Options header set to 'nosniff', the browser may interpret the response body as a different content type than intended. This can result in the incorrect rendering of the content or the execution of malicious code.

3. Compatibility issues: Current (early 2014) and legacy versions of Firefox will use the declared content type (if one is set), rather than performing MIME-sniffing. However, if the declared content type is incorrect or missing, it may still lead to compatibility issues with Firefox and other browsers that follow the same behavior.

Path	Method	Status
/blind-sql-injection	GET	NEW
/business-logic	GET	NEW
/cmd-injection	GET	NEW
/dashboard	GET	NEW
/index.css	GET	NEW
/reflected-xss	GET	NEW
/robots.txt	GET	NEW
/security-misconfig	GET	NEW
/sensitive-data-exposure	GET	NEW
/settings	GET	NEW
/sql-injection	GET	NEW
/xxe	GET	NEW

Cross-Domain JavaScript Source File Inclusion

Criticality: Low CWE ID: 829

Description:

Remediation

To mitigate the vulnerability of Cross-Domain JavaScript Source File Inclusion, the following steps can be taken:

- 1. Implement Content Security Policy (CSP):** Use CSP to restrict the domains from which scripts can be loaded. This can be done by setting the script-src directive to only allow scripts from trusted domains. For example, the following CSP header restricts scripts to be loaded only from the same domain:

```
Content-Security-Policy: script-src 'self';
```

- 2. Use Subresource Integrity (SRI):** Implement SRI to ensure the integrity of the scripts being loaded. SRI allows you to specify a hash of the script file, and the browser will only execute the script if the hash matches. This prevents any tampering or modification of the script file. For example, the following script tag includes the SRI attribute:

```
<script src="https://example.com/script.js" integrity="sha256-abc123..." crossorigin="anonymous"></script>
```

- 3. Avoid using third-party scripts:** Whenever possible, host and serve all scripts from your own domain. This reduces the risk of including malicious scripts from third-party domains.

About

Cross-Domain JavaScript Source File Inclusion occurs when a web page includes one or more script files from a third-party domain. This vulnerability can be exploited by an attacker to execute arbitrary code on the victim's browser, potentially leading to various security risks.

Risks

Including script files from a third-party domain can introduce several risks:

- 1. Code Execution:** An attacker can inject malicious code into the script file, which will be executed in the context of the victim's browser. This can lead to various attacks, such as cross-site scripting (XSS) or data exfiltration.
- 2. Data Leakage:** The third-party script may have access to sensitive information on the web page, such as user credentials or personal data. If the script is compromised, this information

can be leaked to the attacker.

3. Supply Chain Attacks: By including scripts from third-party domains, you are relying on the security practices of those domains. If any of the third-party domains are compromised or have malicious intentions, your website can become a target for supply chain attacks.

It is crucial to implement proper security measures to mitigate the risks associated with Cross-Domain JavaScript Source File Inclusion.

Path	Method	Status
/blind-sql-injection	GET	NEW
/business-logic	GET	NEW
/cmd-injection	GET	NEW
/reflected-xss	GET	NEW
/settings	GET	NEW
/sql-injection	GET	NEW

Server Leaks Version Information via "Server" HTTP Response Header Field

Criticality: Low CWE ID: 200

Description:

Remediation

To remediate this vulnerability, you should modify the server configuration to remove or obfuscate the version information from the "Server" HTTP response header. This can typically be done by adjusting the server configuration file or using a security module or plugin.

Here are some examples of how to remediate this vulnerability for popular web servers:

- 1. Apache HTTP Server:** Edit the httpd.conf file and add the following line to remove the "Server" header:

```
ServerTokens Prod
```

This will set the server token to "Prod" which will hide the version information.

- 2. Nginx:** Edit the nginx.conf file and add the following line to remove the "Server" header:

```
server_tokens off;
```

This will disable the server token and hide the version information.

- 3. Microsoft IIS:** Open the IIS Manager, select the server node, and double-click on the "HTTP Response Headers" feature. Then, click on "Remove" in the "Actions" pane to remove the "Server" header.

Alternatively, you can use the following command in the command prompt to remove the "Server" header:

```
appcmd.exe set config /section:httpProtocol /-  
customHeaders.[name='Server']
```

This will remove the "Server" header from the server responses.

About

The vulnerability "Server Leaks Version Information via "Server" HTTP Response Header Field" occurs when the web/application server includes version information in the "Server" HTTP response header. This information can be easily accessed by attackers and may provide them with valuable insights into the server's software and version.

By knowing the server's software and version, attackers can specifically target vulnerabilities that are known to exist in that particular version. This increases the risk of successful attacks and compromises the security of the web/application server.

Risks

The risks associated with this vulnerability include:

- Increased attack surface:** By leaking version information, the web/application server exposes itself to potential attacks targeting known vulnerabilities in that specific version.
 - Targeted attacks:** Attackers can use the leaked version information to tailor their attacks and exploit vulnerabilities that are known to exist in that version.
 - Reduced security:** The leakage of version information undermines the principle of security through obscurity, making it easier for attackers to gather intelligence and plan their attacks.
- It is important to remediate this vulnerability to prevent attackers from gaining valuable information about the server and reducing the risk of successful attacks.

Path	Method	Status
[ROOT]	GET	NEW
/blind-sql-injection	GET	NEW
/business-logic	GET	NEW
/cmd-injection	GET	NEW
/dashboard	GET	NEW
/dom-xss	GET	NEW
/improper-cert-valid	GET	NEW
/insecure-file-upload	GET	NEW
/logout	GET	NEW
/reflected-xss	GET	NEW
/robots.txt	GET	NEW
/security-misconfig	GET	NEW
/settings	GET	NEW
/sitemap.xml	GET	NEW
/sql-injection	GET	NEW
/stored-xss	GET	NEW